



Project Report

Digital IC Design Academy

Pablo Jose Lopez Mazariegos

Universidad Galileo
Guatemala, 2026

Legal Disclosure

CONFIDENTIAL INFORMATION

The information contained in this presentation is the confidential and proprietary information of Synopsys. You are not permitted to disseminate or use any of the information provided to you in this presentation outside of Synopsys without prior written authorization.

IMPORTANT NOTICE

This presentation may include information related to Synopsys' future product or business plans. Such plans are as of the date of this presentation and subject to change. Synopsys is not obligated to update this presentation or develop the products with the features and/or functionality discussed in this presentation. Additionally, Synopsys' products and services may only be offered and purchased pursuant to an authorized quote and purchase order or a mutually agreed upon written contract.

FORWARD LOOKING STATEMENTS

This presentation may include certain statements including, but not limited to, Synopsys' financial targets, expectations and objectives; business and market outlook, business opportunities, strategies and technological trends; and more. These statements are made only as of the date hereof and subject to change. Actual results or events could differ materially from those anticipated in such statements due to a number of factors. Synopsys undertakes no duty to, and does not intend to, update any statement in this presentation, whether as a result of new information, future events or otherwise, unless required by law.

Agenda

- El diseño: MicroRV8, un CPU de 8 bits propio
- Verification — VCS, Verdi y coverage (~6 min)
 - Testbenches por módulo y del sistema completo
 - Waveforms de las 8 instrucciones requeridas
- Logic Synthesis — Design Compiler (~3 min)
 - 5 variantes de compile_ultra, Formality y DFT
- Physical Synthesis — IC Compiler II (~3 min)
 - Floorplan, place, CTS, route, signoff, StarRC y PrimeTime
- PPA Sweep (~3 min)
- Corrección de PDK y re-ejecución del flujo (~3 min)
- Conclusiones

Delivery Requirements

Qué se pidió y qué entregué

- **Objetivo del proyecto**

- Demostrar el flujo digital completo implementando un procesador multi-ciclo funcional, desde la descripción RTL hasta el diseño físico.
- Interpretar y evaluar los trade-offs de power, performance y area.

- **Entregables cubiertos**

- RTL propio (10 módulos Verilog) + testbenches en SystemVerilog.
- Reportes de síntesis (QoR, power, area, check_design) para 5 variantes de compile_ultra.
- Reportes de layout: floorplan, placement, CTS, routing, DRC/LVS y signoff.
- Caracterización PPA: barrido de 5 periodos de reloj + 2 variantes en el PDK nuevo.
- Scripts reproducibles de cada etapa (dc, fm, icc2, starrc, pt, comparadores, sweep).

- **Diferencia respecto a la plantilla**

- No usé el CPU de referencia: el diseño es un SoC propio con GPIO, UART, PWM y bootloader, previamente validado como softcore en una FPGA Tang Nano 9K, con sus herramientas custom (IDE propio con dissasembler para RISC V, simulación con cocotb, paso a paso con la tang nano)

El proyecto en números

10

módulos Verilog

8

instrucciones de la ISA

5 + 2

variantes de síntesis

5

periodos en el PPA sweep

2

PDKs Sky130

0

violaciones en signoff

Team / Individual Introduction

Quién soy

Pablo José López Mazariegos, soy investigador en Universidad Galileo en el Laboratorio Tesla, me especializo en el desarrollo de sistemas embebidos, software embebido y diseño CAD
Tengo 23 años y estoy en ultimo año de ingeniería en mecatrónica

Me gusta jugar videojuegos y cocinar

Motivación

Ya había diseñado y simulado un CPU de 8 bits y lo había corrido como softcore en una FPGA Tang Nano 9K.
Lo que me faltaba era hacer un layout real con herramientas de la industria y entender qué le pasa al diseño en cada etapa.
Me encanta la electrónica y el tener la oportunidad de aprender el flujo de diseño de ICs fue simplemente increíble y emocionante

Lo que más me interesó del programa

VCS

Compilar y simular RTL + testbenches
con coverage de line, cond, fsm, tgl y branch.

Verdi

Depurar en waveform: modo analog, FSM,
y confirmar el comportamiento señal por señal.

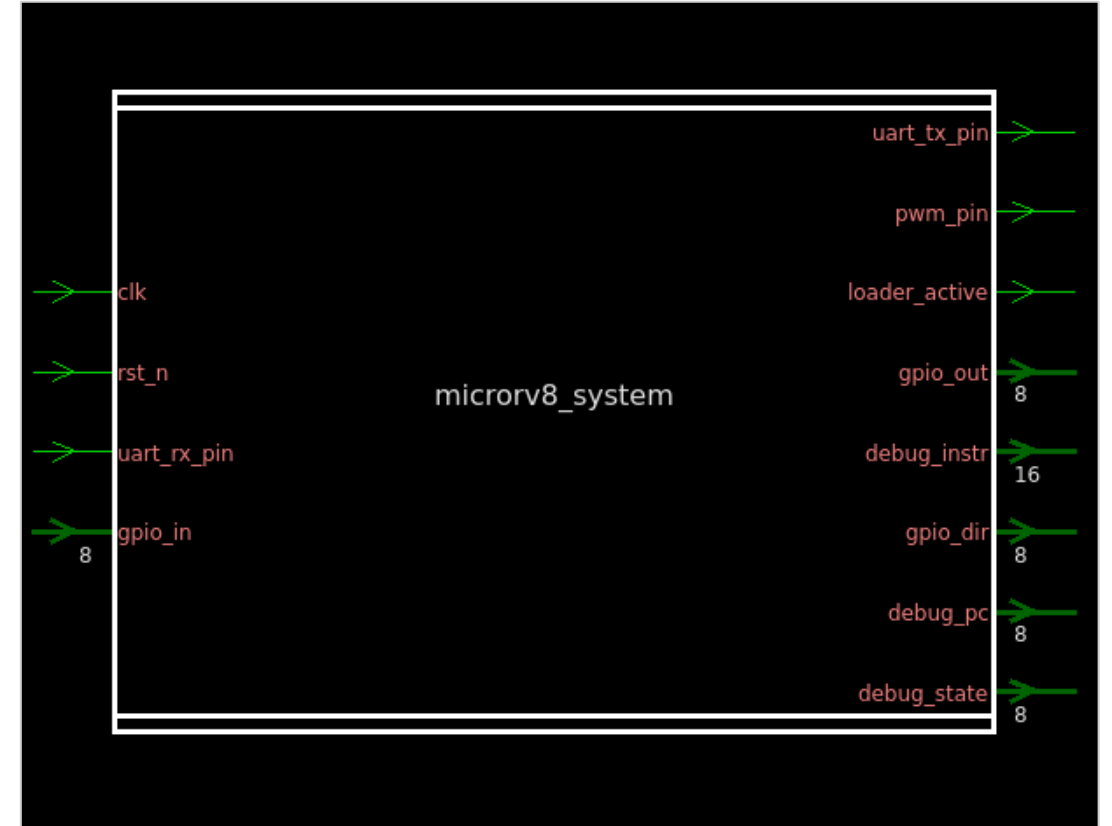
Verilog / SystemVerilog

Los HDL siempre me llamaron la atención desde que los descubri en un curso de la Universidad, poder ver todas las capacidades que tiene verilog y lo que se puede hacer de la mano de herramientas profesionales (Como UVM)

El diseño: MicroRV8

SoC de 8 bits con ISA inspirada en RISC-V, periféricos MMIO y bootloader por UART

- **CPU multi-ciclo — `cpu_core.v`**
 - FSM de 6 estados: FETCH → WAIT → DECODE → EXEC → MEM → WB
 - PC de 9 bits, instruction register de 16 bits
- **ALU de 8 bits — `alu.v`**
 - ADD, SUB, AND, OR, XOR, SLL, SRL, SLT + flags zero/carry/negative
- **Regfile — `regfile.v`**
 - 8 registros de 8 bits; r0 cableado a cero
- **Periféricos memory-mapped**
 - 0x00–0x7F RAM de 128 bytes
 - 0x80–0x82 GPIO_OUT / GPIO_IN / GPIO_DIR
 - 0x83–0x84 UART_TX / UART_STAT
 - 0x85–0x87 PWM_DUTY / PWM_CTRL / PWM_PRESCALER
 - `uart_loader.v` — carga programas por UART en runtime



Vista lógica de `microrv8_system` en Design Compiler

ISA de 16 bits

opcode[15:13] · rd[12:10] · rs1[9:7] · funct3/rs2[6:4] · imm[3:0]

- **Campos de la instrucción**

- opcode — 3 bits, selecciona el tipo de operación
- rd / rs1 / rs2 — registros de 3 bits
- funct3 — selecciona la operación de la ALU
- imm — 4 bits con extensión de signo (ADDI, LOAD, STORE)

- **Del ASM al binario**

- Escribí un disassembler propio en Python que traduce ASM estilo RISC-V a la codificación de mi ISA.
- python3 disassembler.py fib.asm -o salida.hex
- El .hex se carga con \$readmemh en instruction_memory, o por UART con el bootloader.

Ejemplo

Armamos la instrucción ADD r2, r0, r1 suponiendo r0 = 5 y r1 = 3 (estos se pueden cargar previamente con ADDI)

La instrucción en binario queda como 0010100000100000, si la pasamos a hexadecimal: 2820

Bits	15-13	12-10	9-7	6-4	3-1	0
Registro	opcode	rd	rs1	rs2	funct3	Reservado
Valor	001	010	000	001	000	0
Cantidad	3 bits	3 bits	3 bits	3 bits	3 bits	1 bit

Mapa de opcodes

opcode	tipo	operación
000	ALU-I	ADDI rd, rs1, imm
001	ALU-R	ADD/SUB/AND/OR/... rd, rs1, rs2
010	LOAD	rd = MEM[rs1 + imm]
011	STORE	MEM[rs1 + imm] = rd
100	BEQ	salta si rs1 == rs2
101	JAL	rd = PC + 1; PC = target
110	OUT	gpio_out = rs1
111	JUMP	PC = target (incondicional)

Resultado Esperado

ADD r2, r0, r1
r0 = 5
r1 = 3
r2 = r0 + r1
r2 = 5 + 3
r2 = 8

El flujo implementado

De mi RTL en Verilog hasta el layout con timing cerrado, dos veces (PDK original y PDK corregido)

01

RTL

Verilog propio
10 módulos

02

VCS + Verdi

Simulación,
waveforms, coverage

03

Design Compiler

5 variantes de
compile_ultra

04

Formality + DFT

Equivalencia formal
y scan chains

05

IC Compiler II

Floorplan, place,
CTS, route

06

**StarRC +
PrimeTime**

Extracción RC
y STA con SPEF

Tecnología y herramientas

- PDK: SkyWater Sky130 HD (sky130_fd_sc_hd)
- Corners: ff_100C_1v95 y ss_100C_1v40
- Design Compiler / Formality / ICC2 / StarRC / PrimeTime
- VCS + Verdi con coverage line+cond+fsm+tgl+branch
- Periodo objetivo base: 20 ns (50 MHz)

Por qué el flujo se corrió dos veces

La primera pasada cerró, pero arrastraba errores del PDK original, entre ellos VSS y VDD aparecían sin conexión

Migré todo a una versión actualizada del PDK Sky130 y volví a correr el flujo con el mismo RTL.

Los resultados de ambas pasadas se comparan al final de la presentación.

Verification

VCS · Verdi · Coverage — la parte que mas me gusto

Metodología de verificación

Un testbench por módulo antes de integrar, y luego el sistema completo con programas reales

- **Estrategia**

- Cada módulo (alu, regfile, data_memory, uart, pwm) se compila y simula por separado, con su propio simv y su propio .vdb.
- Solo después se integra y se simula microrv8_system cargando programas .hex reales.

- **Qué da VCS**

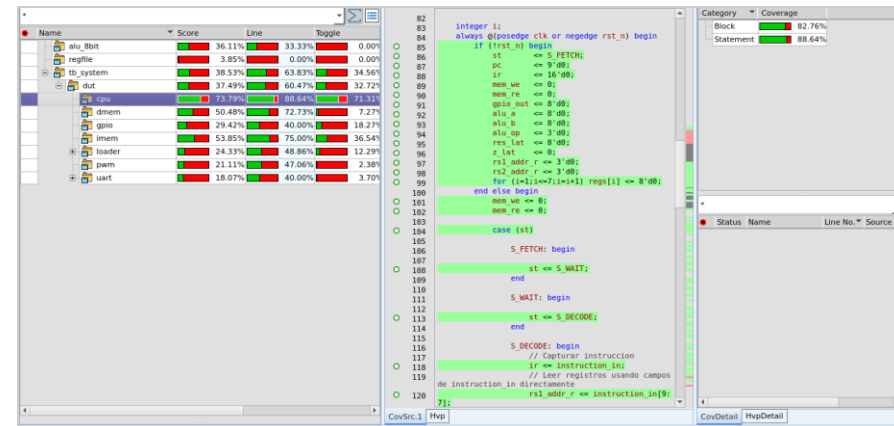
- -kdb -debug_access+all para poder abrir todo en Verdi.
- -cm line+cond+fsm+tgl+branch para medir cobertura por módulo.
- +define+PROGRAM_HEX para cambiar el programa sin tocar el RTL.

- **Qué da Verdi**

- Modo analog para ver el duty del PWM como una señal real.
- Navegación de FSM para seguir
FETCH→WAIT→DECODE→EXEC→MEM→WB.
- Vista de coverage por módulo, línea a línea sobre el código.

Line	Toggle	FSM	Condition	Branch	Assert
*					
	Variable	Type	Coverage	Display	
	▲ a[7:0]	signal	100.00%	◆◆◆◆◆◆◆◆	
	▲ b[7:0]	signal	100.00%	◆◆◆◆◆◆◆◆	
	▲ carry	signal	100.00%	◆	
	▲ negative	signal	100.00%	◆	
	▲ op[2:0]	signal	100.00%	◆◆◆	
	▲ result[7:0]	signal	100.00%	◆◆◆◆◆◆◆◆	
	▲ zero	signal	100.00%	◆	

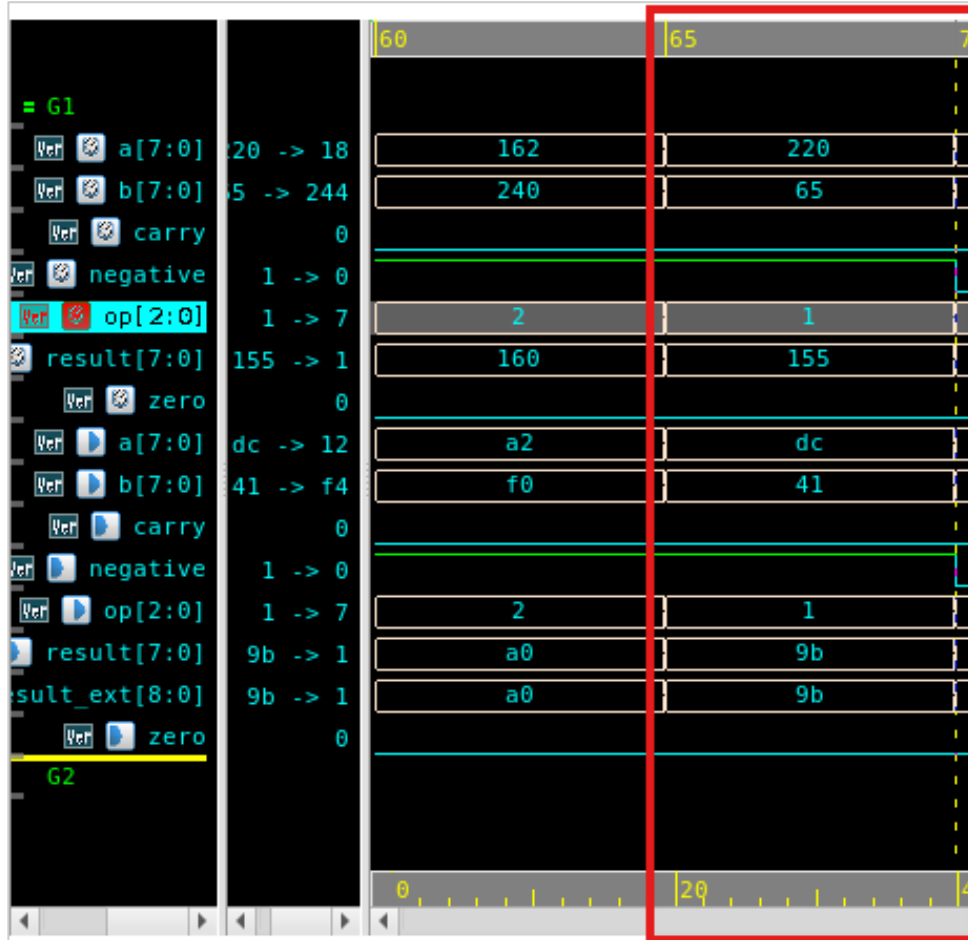
Coverage de ALU



Coverage del sistema en Verdi: score por módulo sobre el RTL

Verification — ALU

tb_alu.sv · simv_alu · alu.fsdb — comprobación operación por operación



Qué muestra la waveform

En el cursor se ejecuta $op = 1$ (SUB) con $a = 220$ y $b = 65$.

$result = 220 - 65 = 155$, y la waveform marca exactamente 155.

La flag zero se mantiene en 0, que es lo correcto porque el resultado no es cero. carry y negative también son consistentes con la resta sin préstamo.

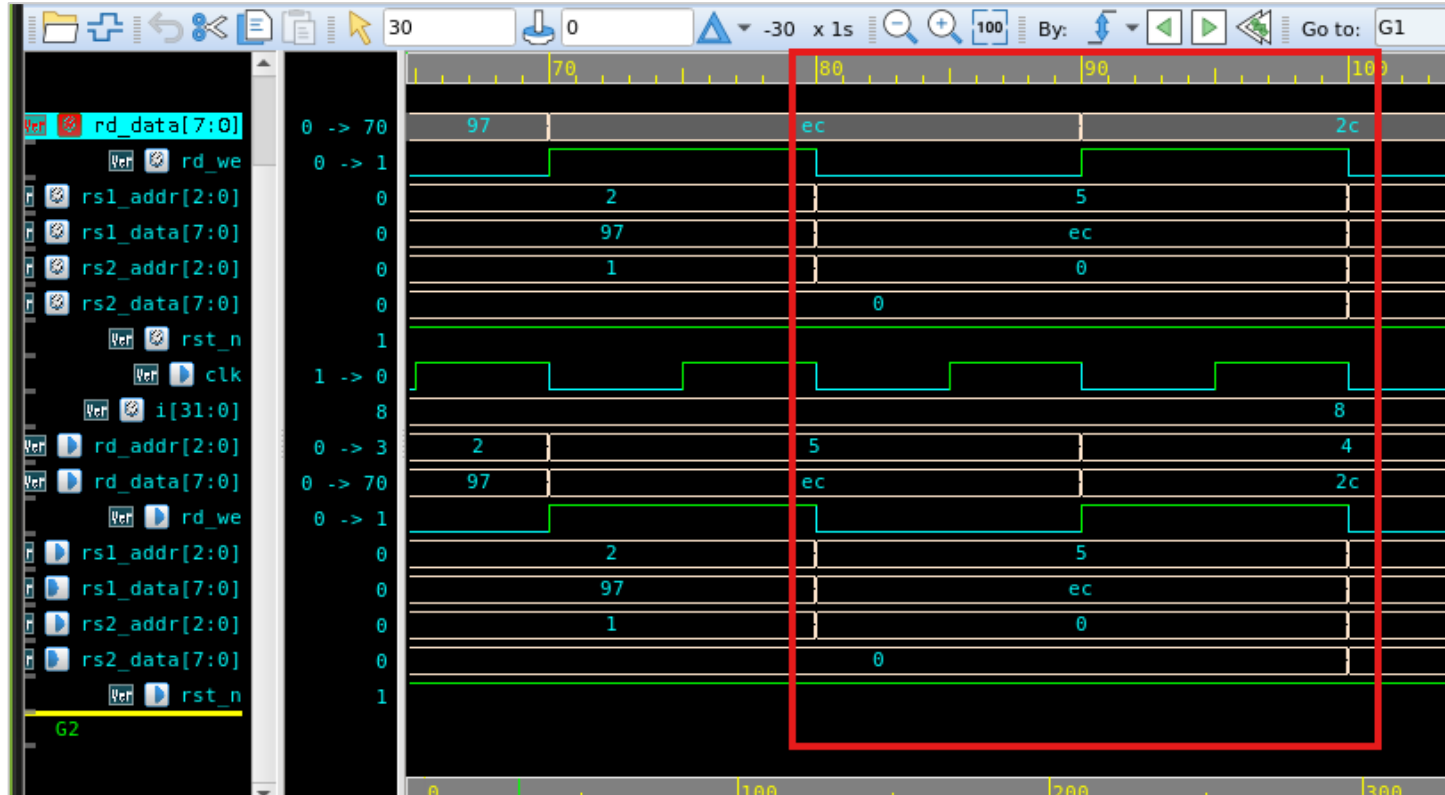
Por qué empecé por aquí

La ALU es puramente combinacional: si falla aquí, todo lo demás falla y puede empeorar todo

Verificarla después con coverage me hizo ver que si era confiable mi rtl de ALU

Verification — Regfile

Escritura contra lectura, y aislamiento entre registros



Lectura de la waveform

Se escribe 0xEC en el registro 5: $rd_addr = 5$, $rd_data = ec$, $rd_we = 1$.

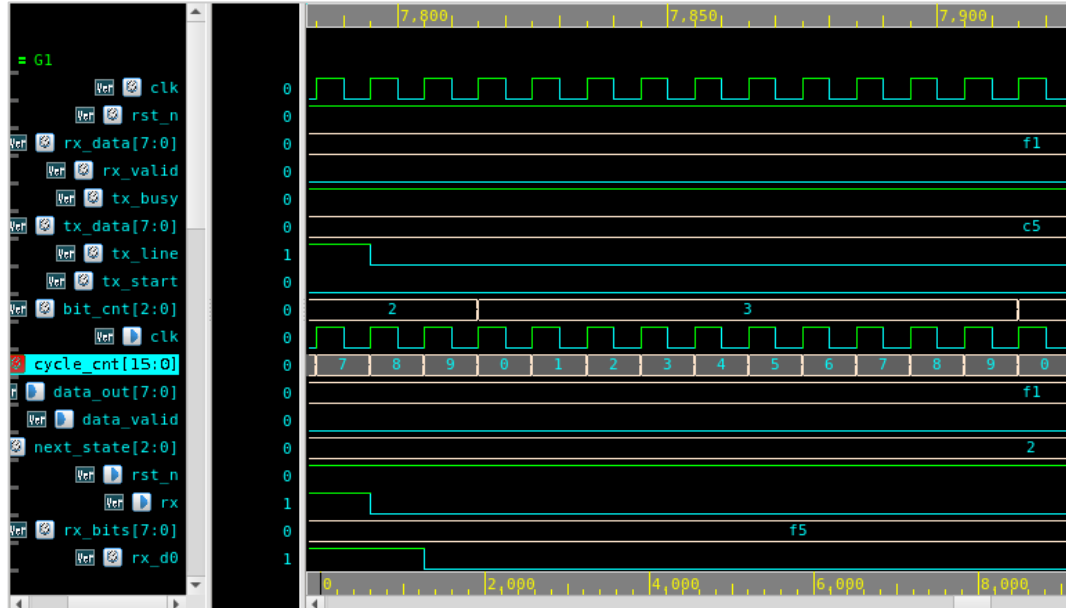
En el ciclo siguiente se lee ese mismo registro por rs1: $rs1_addr = 5 \rightarrow rs1_data = ec$.

Coincide exactamente con lo escrito: la escritura llegó al registro correcto.

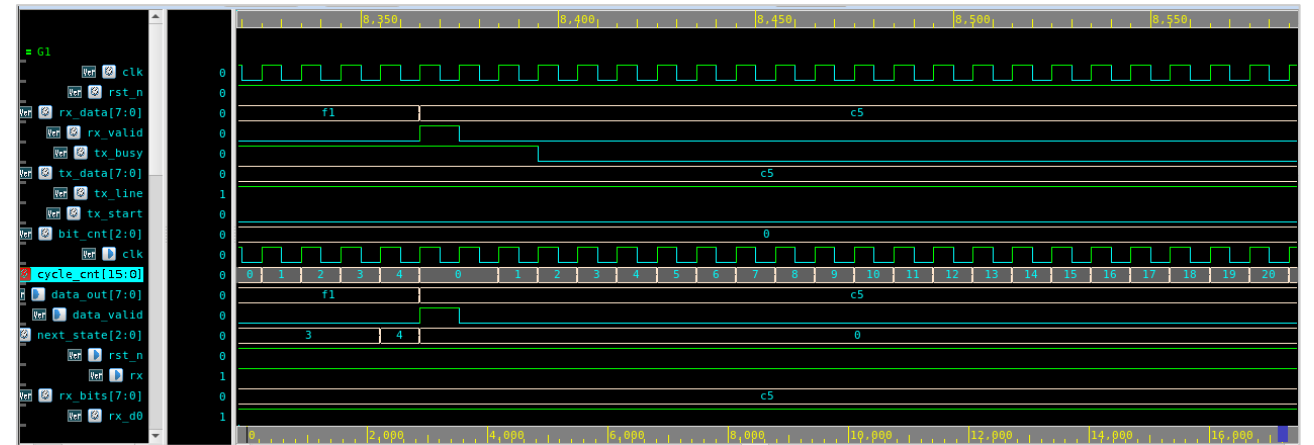
Al mismo tiempo rs2_addr apunta al registro 0, que nunca se escribió, y rs2_data se mantiene en 0. Eso confirma que la escritura no contaminó el resto del banco.

Verification — UART

Loopback TX → RX: el byte 0xC5 viaja completo y llega sin corrupción



Zoom al final de la recepción del byte



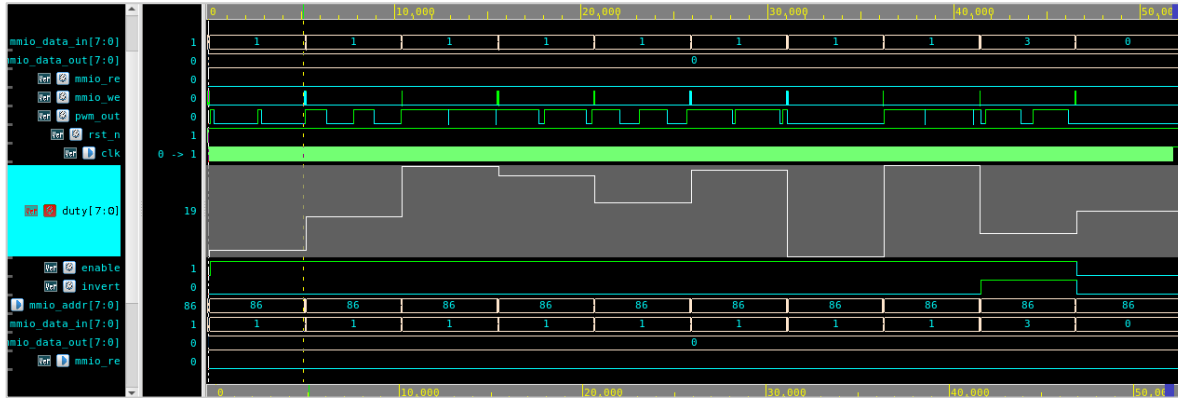
Trama completa: start, 8 bits de datos y stop

Análisis

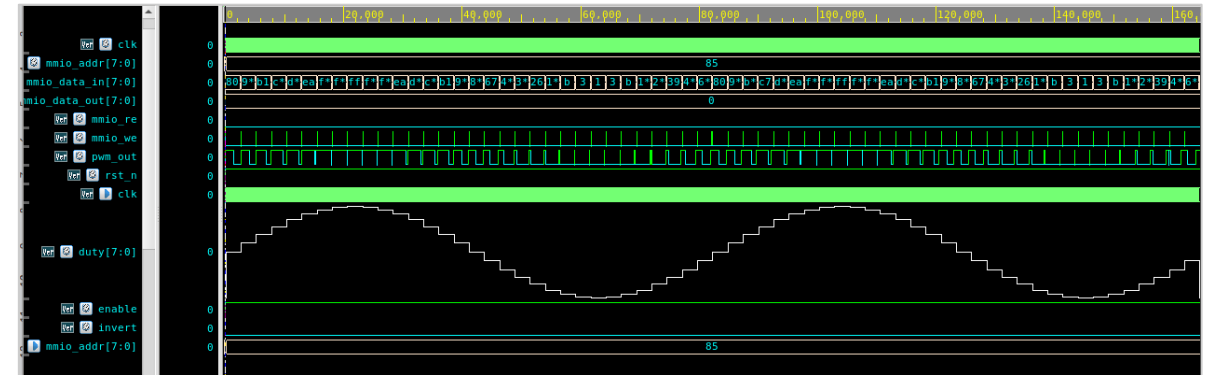
- Cuando `cycle_cnt` llega a 0 tras el último bit de dato, la FSM del receptor avanza de estado 3 a estado 4.
- Se genera el pulso `rx_valid`, y tanto `rx_bits` como `data_out` pasan de 0xF1 (byte anterior) a 0xC5.
- 0xC5 coincide exactamente con `tx_data` = c5, así que el loopback transmitió el byte sin corrupción.
- La caída simultánea de `tx_busy` confirma que el lado TX también terminó su transmisión.

Verification — PWM y generador de funciones

El modo analog de Verdi convierte el registro DUTY en una forma de onda visible



tb_pwm.sv — el duty programado coincide con los tiempos en alto y bajo del pin



tb_pwm_wave.sv — DUTY siguiendo una fórmula senoidal

El experimento

En vez de valores random, tb_pwm_wave.sv programa el registro DUTY del PWM siguiendo una fórmula: senoidal, triangular, diente de sierra y cuadrada.

Cada muestra se sostiene un periodo completo (256 ciclos con prescaler = 1), así que mmio_data_in traza la forma de onda exacta cuando se ve en modo Analog.

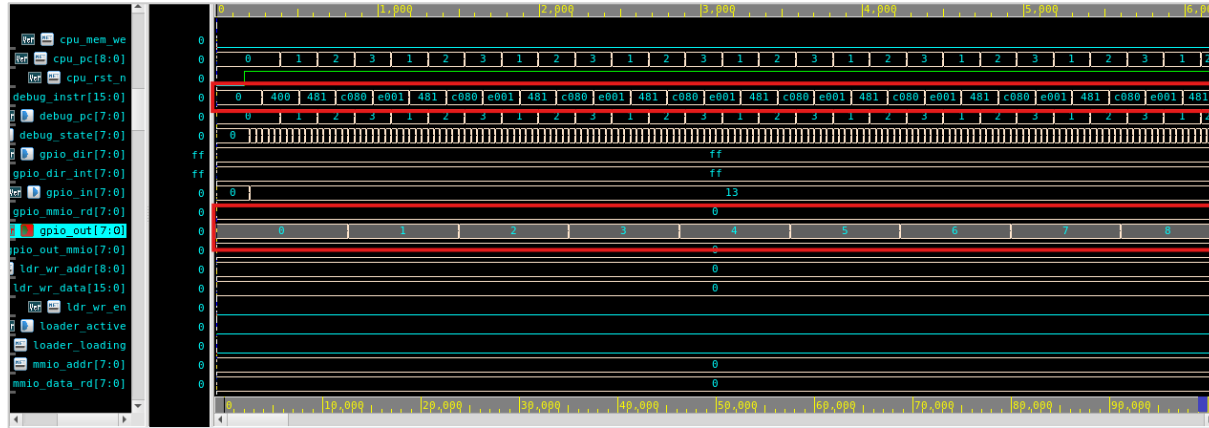
Lo que aprendí aquí

Fue puramente educativo y curioso, pero me mostró dos cosas:

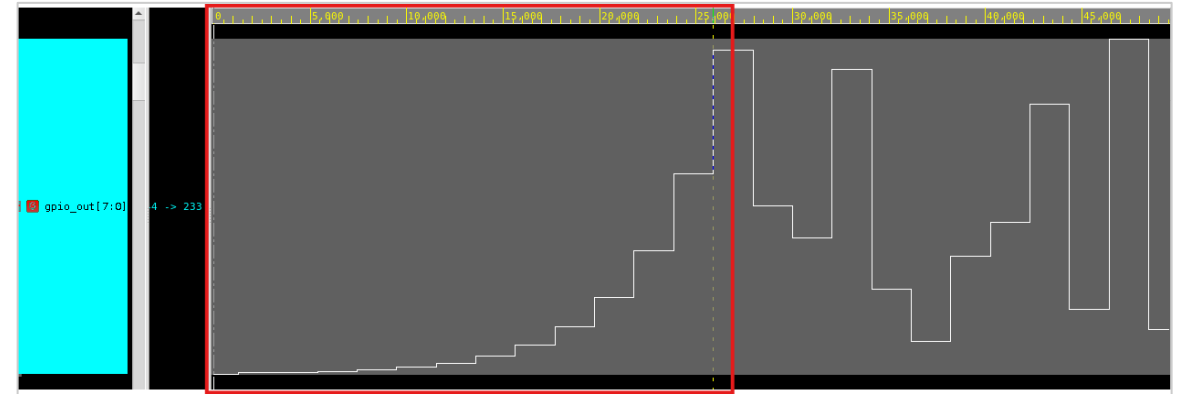
- SystemVerilog es capaz de hacer cálculos relativamente complejos dentro del testbench, no solo aplicar estímulos.
- El coverage del PWM sale más bajo que el del resto, y eso es esperable: la señal es estable por diseño, no hace toggle a cada rato.
Un número bajo no siempre es un problema.

Verification — sistema completo

ejecutando programas reales cargados como .hex



count.hex — contador simple, las instrucciones se repiten y gpio_out sube



salida.hex — Fibonacci en modo analog sobre gpio_out

Contador cargado en la ROM

```
rom[0] 0x0400 ADDI r1, r0, 0    r1 = 0
rom[1] 0x0481 ADDI r1, r1, 1    r1++
rom[2] 0xC080 OUT  r1           gpio_out = r1
rom[3] 0xE001 JUMP 1            loop
```

Se carga con +define+PROGRAM_HEX, sin tocar el RTL.

Fibonacci desde ASM RISC-V

14 instrucciones ensambladas con mi disassembler (Python).

La secuencia coincide hasta 233. Al intentar llegar a 377 hace overflow, que es exactamente lo esperado en un datapath de 8 bits.

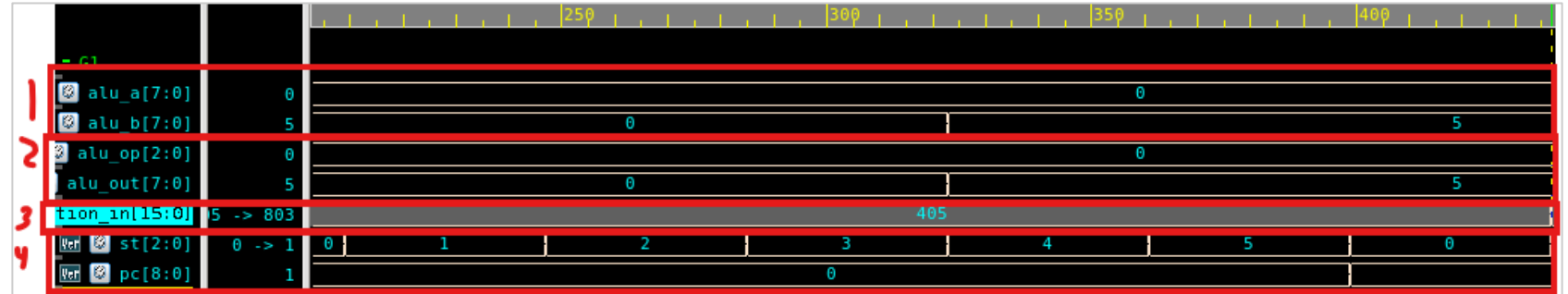
Esto valida las dos cosas a la vez: el CPU ejecuta correctamente y la cadena ASM → hex → memoria

Waveforms de las 8 instrucciones requeridas (1/3)

Programa isa8.hex — señales: alu_a, alu_b, alu_op, alu_out, instruction_in, pc y st (estado)

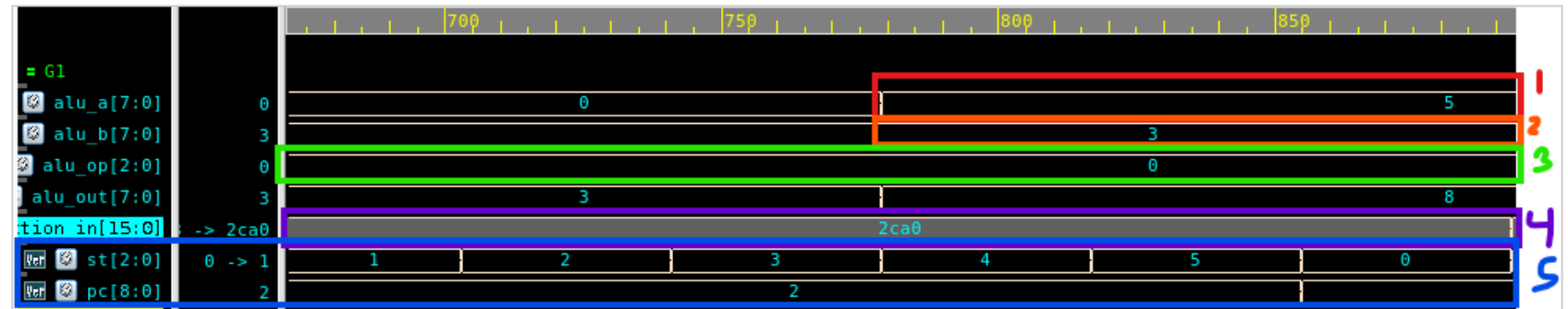
ADDI · pc = 0 · instr = 0x0405

ADDI r1, r0, 5. El estado recorre FETCH → WAIT → DECODE → EXEC → MEM → WB. En WB el registro r1 pasa de 0 a 5: $r1 = r0 + 5 = 5$.



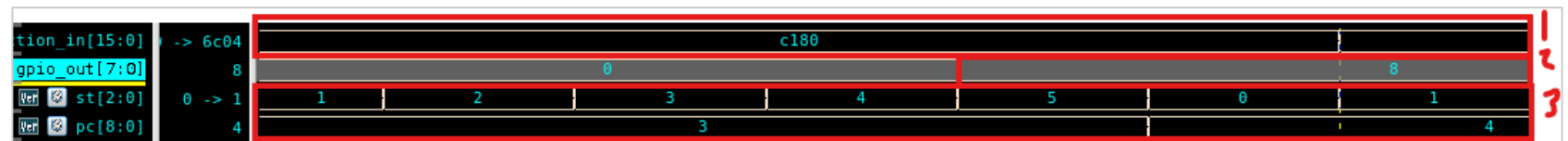
ADD · pc = 2 · instr = 0x2ca0

ADD r3, r1, r2 con r1 = 5 y r2 = 3 ya cargados. En EXEC la ALU suma ambos registros y en WB r3 toma el valor 8.



OUT · pc = 3 · instr = 0xc180

OUT r3. En el estado MEM el valor de r3 se copia al puerto de salida: gpio_out pasa a 8, confirmando que el dato sale del CPU.

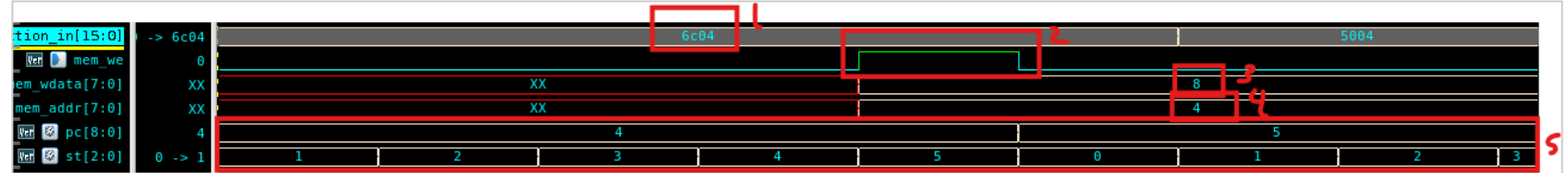


Waveforms de las 8 instrucciones requeridas (2/3)

Para memoria se añaden a la waveform mem_we, mem_re, mem_addr, mem_wdata y mem_rdata

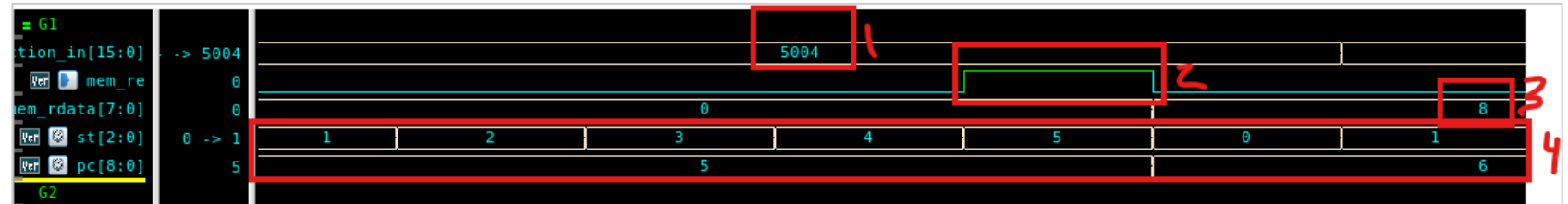
STORE · pc = 4 · instr = 0x6c04

STORE r3, r0, 4. La dirección se calcula como $r0 + 4 = 4$. En MEM se activa $mem_we = 1$, $mem_addr = 4$ y $mem_wdata = 8$, así que $RAM[4] = 8$.



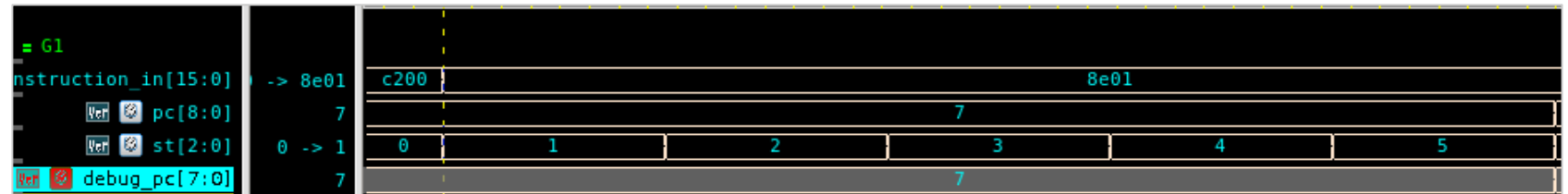
LOAD · pc = 5 · instr = 0x5004

LOAD r4, r0, 4. Se activa $mem_re = 1$ sobre la dirección 4 y mem_rdata devuelve el 8 guardado por STORE. En WB $r4 = 8$: roundtrip store/load cerrado.



BEQ · pc = 7

BEQ r3, r4 con $r3 = r4 = 8$. El salto se toma y el pc pasa de 7 a 9 sin ejecutar pc = 8. La prueba: r5 se queda en 0 porque la instrucción que lo cargaba nunca corre.

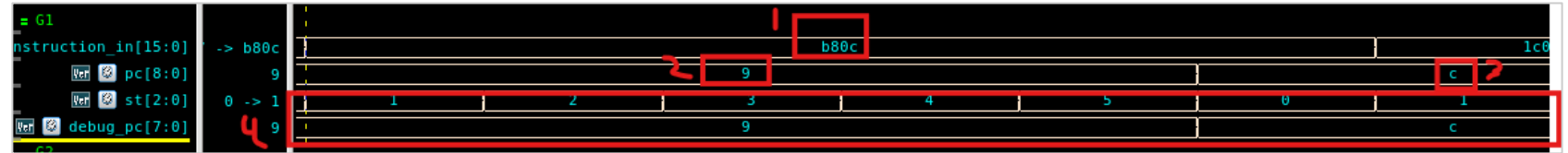


Waveforms de las 8 instrucciones requeridas (3/3)

Los saltos se leen con pc, instruction_in y el estado del CPU

JAL · pc = 9

JAL r6, sub. El registro r6 guarda la dirección de retorno (pc + 1) y el pc salta a la subrutina. Al terminar se regresa usando la dirección guardada en r6.



Discussion — ¿cómo aseguro que el RTL está libre de bugs?

Lo que hice, lo que encontré y lo que haría con más tiempo

Lo que hice

- Testbench propio por módulo antes de integrar nada.
- Coverage line + cond + fsm + tgl + branch en cada simulación.
- Programas reales (contador, Fibonacci, isa8, pwm) en vez de solo estímulos random
- Verificación formal con Formality: 9,630 compare points, 0 failing.
- Chequeo cruzado: el mismo RTL ya corría como softcore en una FPGA Tang Nano 9K.

Bug real encontrado

- La decodificación de instrucciones tipo R estaba mal: rs2 y funct3 se leían de los bits equivocados.
- Se detectó porque ADD daba un resultado que no correspondía a los operandos en la waveform.
- Corregido en `cpu_core.v`: `rs2 = ir[6:4]` y `funct3r = ir[3:1]`.
- Sin la waveform de Verdi, este bug se habría propagado hasta el layout.

Que haría con mas tiempo

- Assertions sobre todo el rtl
- Testbench con estímulos aleatorios restringidos y scoreboard contra un modelo de referencia (usar tasks y UVM).
- Comprender mas acerca de STARRC, PT y PC
- Intentar hacer un RISC V real.

Logic Synthesis — cinco variantes de compile_ultra

Mismo RTL, misma SDC, mismo target de 20 ns: solo cambia la estrategia de optimización

- **Las cinco corridas**

- no_autoungroup — sin deshacer la jerarquía
- gate_clock — con clock gating insertado
- retime — con retiming de registros
- no_boundary_optimization — sin optimizar a través de las fronteras de módulo
- incremental — gate_clock más una pasada de refinamiento

- **Comparación automatizada**

- Escribí comparadores en bash que hacen grep de area, slack y power sobre los reportes de cada variante y los tabulan.
- Cinco comparadores: general, check_design, QoR, timing y area.

```
chmod +x scripts/comparador*.sh
./scripts/comparador.sh > DC_results/comparacion.txt
./scripts/comparador_qor.sh > ../comparacion_qor.txt
./scripts/comparador_timing.sh > ../comparacion_timing.txt
```

Todas las variantes cierran timing con el mismo slack (~7.33 ns) y el mismo endpoint crítico. La decisión, entonces, no se toma por timing: se toma por potencia y area.

comparacion.txt ~/FodiseguaPJLM/Reze/DC_results			
VARIANTE	AREA_TOTAL	WNS	POWER_TOT
no_autoungroup	337884.051083	7.33	54.9301 mW
gate_clock	267492.790265	7.35	3.9961 mW
retime	337795.215884	7.33	54.9334 mW
no_boundary_optimization	337895.311884	7.33	54.9358 mW
incremental	267597.891061	7.35	3.9572 mW

comparacion.txt — area, WNS y potencia total de las cinco variantes

comparacion_timing.txt ~/FodiseguaPJLM/Reze/DC_results			
VARIANTE	SLACK	ARRIVAL	STARTPOINT
no_autoungroup	7.33	2.27	
-2.27 gpio/gpio_out_reg[0]		gpio_out[0]	
gate_clock	7.35	2.25	
-2.25 gpio/gpio_out_reg[0]		gpio_out[0]	
retime	7.33	2.27	
-2.27 gpio/gpio_out_reg[0]		gpio_out[0]	
no_boundary_optimization	7.33	2.27	
-2.27 gpio/gpio_out_reg[0]		gpio_out[0]	
incremental	7.35	2.25	
-2.25 gpio/gpio_out_reg[0]		gpio_out[0]	

comparacion_timing.txt — mismo endpoint crítico en todas: gpio/gpio_out_reg[0]

Logic Synthesis — se elige: gate_clock

La variante mas optima en todo sentido

3.996 mW

potencia total
(vs 54.93 mW de las base)

13.7×

menos consumo
que no_autoungroup

267,492

unidades de area
19,741 celdas (vs 27,502)

7.35 ns

de slack sobre 20 ns
0 paths violando

Por qué gate_clock

- El clock gating corta el reloj hacia los ~9,500 flip-flops de las memorias, que rara vez cambian de valor. Ahí está el 93 % del ahorro.
- Descarté incremental: es gate_clock más una pasada de refinamiento (37.94 s de compilación) y no aporta mejora medible.
- Descarté retime: tiene el check_design más limpio (9 warnings vs 30), pero mantiene el consumo en ~54.93 mW.

```
*****
Report : qor
Design : microrv8_system
Version: W-2024.09-SP3
Date   : Wed Jul 29 12:58:17 2026
*****
Information: Timer using 'CRPR'. (TIM-050)

Scenario          'functional_worst'
Timing Path Group '**in2reg_default**'

-----
Levels of Logic:              5
Critical Path Length:         10.21
Critical Path Slack:          7.65
Critical Path Clk Period:     20.00
Total Negative Slack:         0.00
No. of Violating Paths:       0
-----
```

report_qor de la variante gate_clock: 0 violating paths, TNS = 0

Formality y DFT

Equivalencia formal entre RTL y netlist, e inserción de scan chains

Formality — verificación formal

Reference: r:/WORK/microrv8_system
Implementation: i:/WORK/microrv8_system

0. Guid. 1. Ref. 2. Impl. 3. Setup 4. Match 5. Verify 6. Debug

iling Points Passing Points Aborted Points Unverified Points Probe Points Analyses Loops

er filter expression

Type	Reference	r:Size	Implementation	i:Size	+/-
1 DFF	cpu/alu_a_reg[0]		cpu/alu_a_reg[0]		
2 DFF	cpu/alu_a_reg[1]		cpu/alu_a_reg[1]		
3 DFF	cpu/alu_a_reg[2]		cpu/alu_a_reg[2]		
4 DFF	cpu/alu_a_reg[3]		cpu/alu_a_reg[3]		
5 DFF	cpu/alu_a_reg[4]		cpu/alu_a_reg[4]		
6 DFF	cpu/alu_a_reg[5]		cpu/alu_a_reg[5]		
7 DFF	cpu/alu_a_reg[6]		cpu/alu_a_reg[6]		
8 DFF	cpu/alu_a_reg[7]		cpu/alu_a_reg[7]		
9 DFF	cpu/alu_b_reg[0]		cpu/alu_b_reg[0]		

its: 9630 Display names: Original Mapped Analyze Analyze Selected Points Get Loop Data

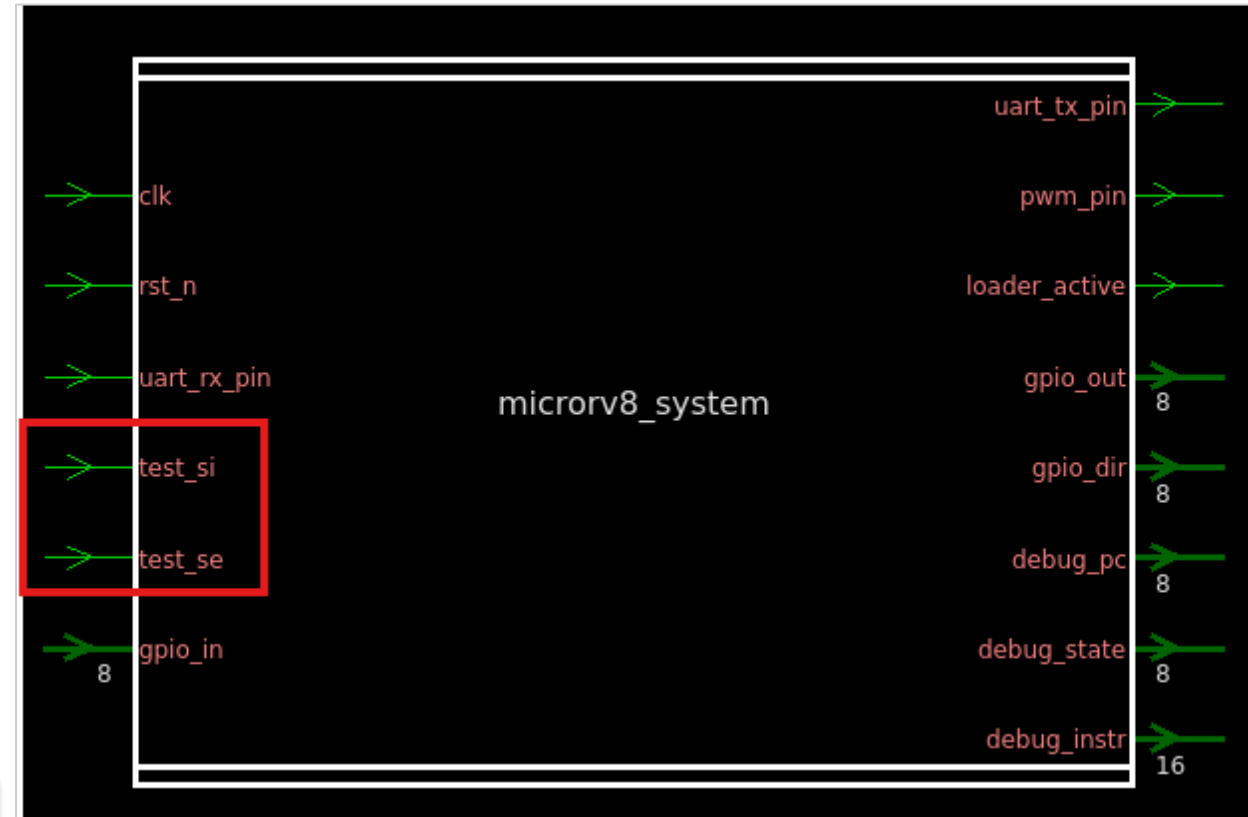
9630 passing compare points

Matched Compare Points	BBPin	Loop	BBNet	Cut	Port	DFF	LAT	TOTAL
Passing (equivalent)	0	0	0	0	51	9579	0	9630
Failing (not equivalent)	0	0	0	0	0	0	0	0

- Referencia: el RTL original. Implementación: mapped.v de gate_clock, usando su .svf.
- 9,630 compare points (9,579 DFF + 51 puertos): todos passing, 0 failing.
- Confirma que compile_ultra con clock gating no cambió la función lógica del diseño.

Formality cierra con que sigue siendo la misma lógica original de mis rtl después de todos los cambios en la compilación

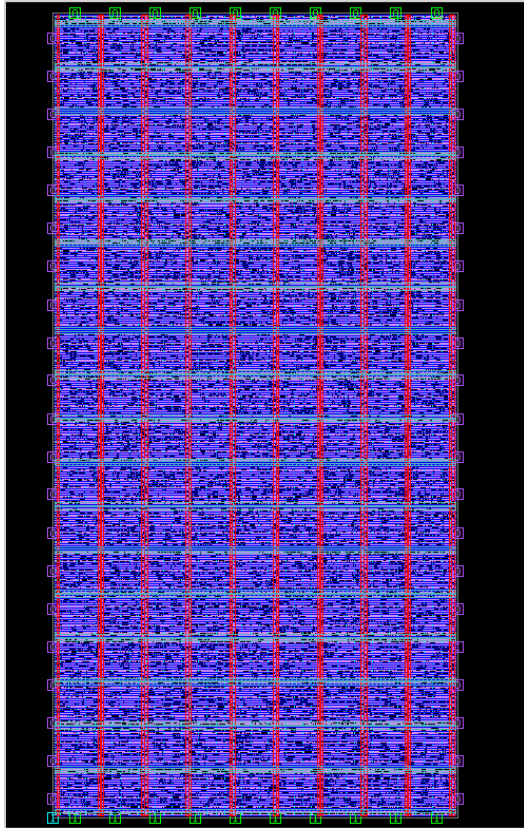
DFT — inserción de scan



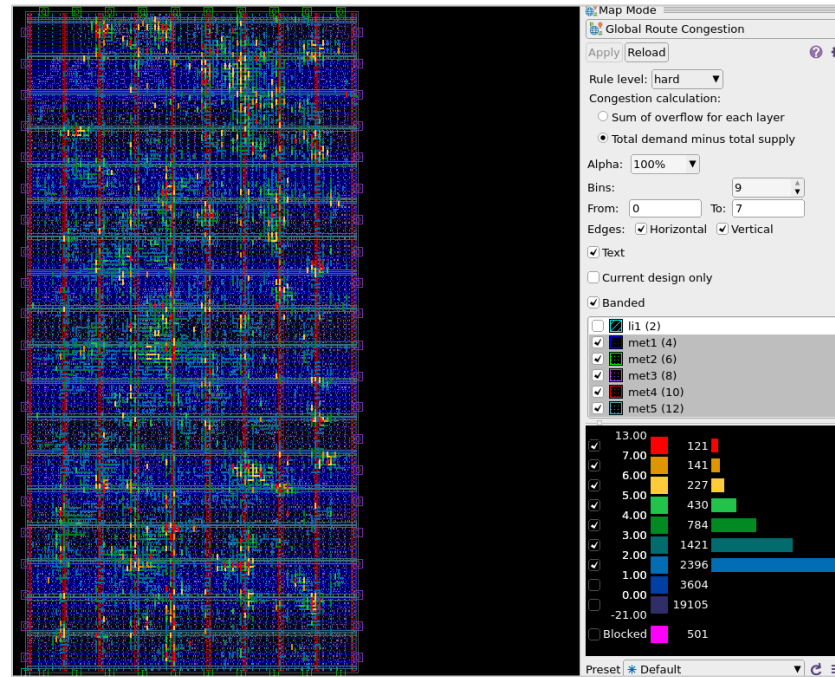
- Aparecen los puertos nuevos test_si y test_se en el top del diseño.
- Reportes de scan path, señales de DFT, timing, QoR y area.

Physical Synthesis — floorplan y placement

IC Compiler II sobre el netlist de gate_clock, con las NDM de Sky130



Floorplan



Congestión de global route después de place_opt

Lo que hace cada script

run_fp.tcl

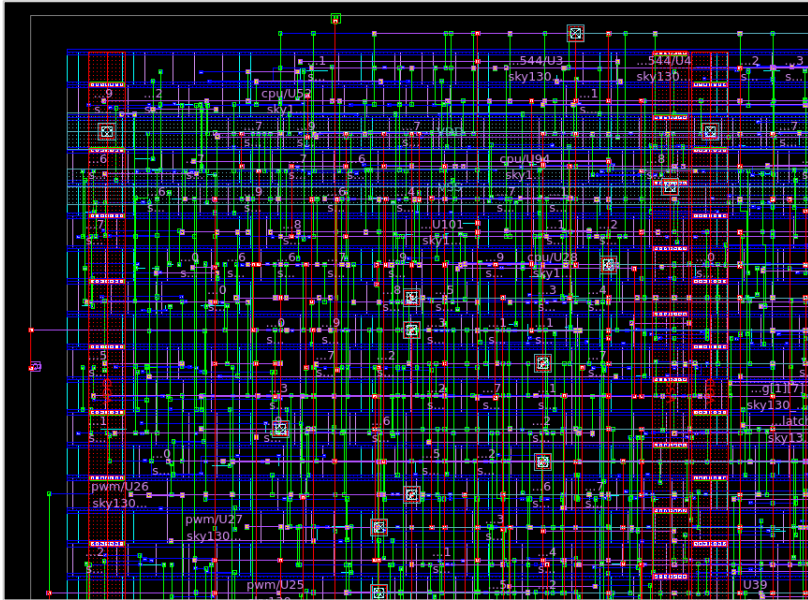
Crea la lib de ICC2 referenciando la NDM de Sky130, lee mapped.v, hace initialize_floorplan con el core utilization, coloca pines con place_pins y lee la SDC que salió de DC.

run_place.tcl

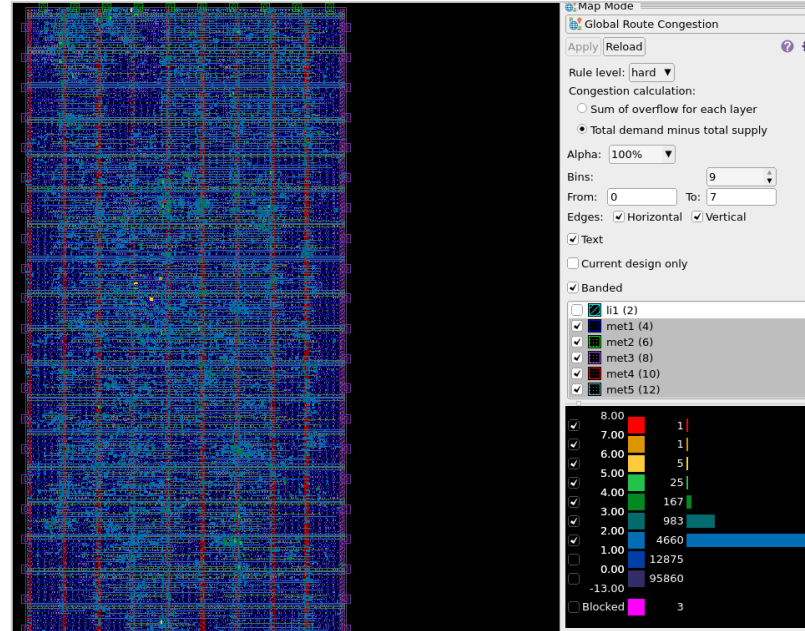
Copia el bloque de floorplan para no perderlo, corre place_opt, valida con check_legality y reporta QoR y utilization.

Physical Synthesis — CTS y routing

clock_opt construye el árbol de reloj, y el ruteo completo cierra sin violaciones de DRC



Post-CTS: buffers e inversores del reloj insertados



Congestión post-route: mejora clara frente a post-place

```
[DRC CHECK] Elapsed real time: 0:00:29
[DRC CHECK] Elapsed cpu time: sys=0:00:00 usr=0:00:29 total=0:00:30
[DRC CHECK] Stage (MB): Used 0 Alloctr 0 Proc 124
[DRC CHECK] Total (MB): Used 146 Alloctr 147 Proc 4728
Warning: No antenna rules defined, Skip antenna analysis. (ZRT-309)

DRC-SUMMARY:
@@@@@@ TOTAL VIOLATIONS = 0

Information: The app option "route.detail.report_soft_drc_separately" will be on
-by-default in future release. (ZRT-807)
@@@@@@ TOTAL SOFT VIOLATIONS = 0
```

check_routes: 0 violaciones DRC, 0 soft violations

9,579

sinks en el clock tree

4

niveles de árbol

5.69 ns

WNS post-CTS

5.61 ns

WNS post-route

0

violaciones de DRC

Signoff — DRC/LVS, StarRC y PrimeTime

Extracción de parásitos y análisis estático de timing sobre el netlist final

StarRC — extracción RC

- Apunta al netlist y layout finales del signoff (GDS + DEF).
- TCAD_GRD_FILE y mapping file del PDK Sky130 con rutas absolutas: así evité el error SX-3012 que me apareció en el laboratorio.
- Dos corners de extracción definidos en el corners file: nominal a 25 °C y nominal_40 a 40 °C. StarRC genera un SPEF por corner.

PrimeTime — STA con parásitos

- Lee el Verilog final, la SDC y el SPEF con read_parasitics.
- Genera reportes de setup, hold, constraints y analysis coverage.
- Aquí aparecieron paths con slack negativo que en ICC2 no se veían: los parásitos reales cuestan.

```
clock clk (rise edge)                2.00      0.00      0.00
clock network delay (ideal)            0.00      1.60      1.60
cpu/pc_reg[6]/CLK (sky130_fd_sc_hd__dftrtp_1) 2.00      0.00      1.60 r
cpu/pc_reg[6]/Q (sky130_fd_sc_hd__dftrtp_1) 0.02      0.38      1.96 & 3.56 r
cpu/ctmTdsLR_1_3336/X (sky130_fd_sc_hd__clkdllybuf4s50_2) 0.31      3.57      3.89 & 7.45 r
imem/U499/Y (sky130_fd_sc_hd__nor2_1) 0.02      0.76      1.58 & 9.03 f
imem/U501/Y (sky130_fd_sc_hd__nand2_1) 0.09      1.69      1.62 & 10.65 r
imem/ctmTdsLR_1_1773/Y (sky130_fd_sc_hd__nor2_2) 0.11      0.94      1.33 & 11.97 f
imem/ropt_mt_inst_3438/X (sky130_fd_sc_hd__clkdllybuf4s50_1) 0.17      2.98      3.63 & 15.60 f
imem/U3320/Y (sky130_fd_sc_hd__a22oi_1) 0.01      1.16      1.92 & 17.52 r
imem/U3324/Y (sky130_fd_sc_hd__nand4_1) 0.06      2.58      2.51 & 20.03 f
imem/U3340/Y (sky130_fd_sc_hd__nor4_1) 0.04      4.12      4.37 & 24.39 r
imem/U3341/Y (sky130_fd_sc_hd__nand4_1) 0.06      2.87      3.71 & 28.11 f
imem/U3342/Y (sky130_fd_sc_hd__a22oi_1) 0.01      1.10      1.87 & 29.98 r
imem/U3514/Y (sky130_fd_sc_hd__nand2_1) 0.11      2.45      2.40 & 32.38 f
imem/data_out_reg[7]/D (sky130_fd_sc_hd__dfxtp_4) 2.46      0.02 & 32.39 f
data arrival time                      32.39

clock clk (rise edge)                0.00      20.00     20.00
clock network delay (ideal)            0.00      0.00      20.00
imem/data_out_reg[7]/CLK (sky130_fd_sc_hd__dfxtp_4) 20.00
clock reconvergence pessimism           1.00      21.00
clock uncertainty                       -2.00     19.00
library setup time                     -1.54     17.46
data required time                     17.46

-----
data required time                      17.46
data arrival time                      -32.39
-----
slack (VIOLATED)                       -14.93
```

report_timing en PrimeTime: path crítico con SPEF cargado

Qué faltó y por qué

El paso siguiente era PrimeTime ECO (Prime Closure): cargar los dos escenarios con read_eco_session, medir WNS/TNS por corner, calcular los fixes y escribir el ECO con write_changes -format icc2tcl para aplicarlo en ICC2.

Dejé el script fix_all.tcl y run_pc.sh listos y probados hasta el punto de ejecución, pero no había licencia de ECO disponible en el laboratorio.

Es el único paso del flujo que quedó pendiente por razones de herramienta, no de diseño. (y un poco de falta de comprensión de los temas de STARRC, PT y PC)

PPA Sweep

Barrido de periodo de reloj de 20 ns a 8 ns sobre la variante gate_clock

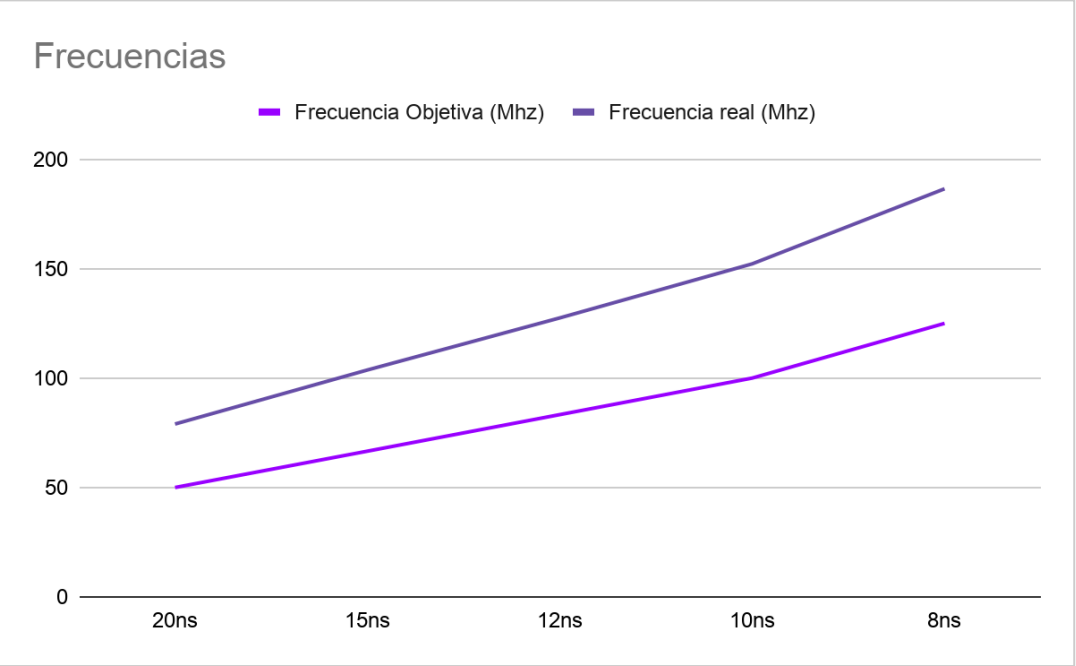
Periodo	F objetivo (MHz)	F efectiva (MHz)	WNS (ns)	Area	Potencia (mW)
20 ns	50.0	79.0	7.35	267,493	3.9961
15 ns	66.6	103.7	5.36	267,380	4.4172
12 ns	83.3	127.5	4.16	267,373	4.8507
10 ns	100.0	152.2	3.43	267,373	5.2843
8 ns	125.0	186.5	2.64	265,629	5.9354

Expectativa vs. resultado

- Esperaba que area subiera al disminuir el periodo
- Esperaba que la potencia subiera de forma lineal con la frecuencia, el clock gating ayuda a que se mantenga baja (supongo?)
- El WNS (Worst Negative Slack) positivo en los cinco puntos confirma que incluso a 125 MHz

```
[synuser18@synopsys2 Reze]$ cd ~/FodiseguaPJLM/Reze
[synuser18@synopsys2 Reze]$ chmod +x scripts/ppa_sweep.sh
[synuser18@synopsys2 Reze]$ ./scripts/ppa_sweep.sh
===== periodo 20 ns =====
===== periodo 15 ns =====
===== periodo 12 ns =====
===== periodo 10 ns =====
===== periodo 8 ns =====
PERIODO      AREA      WNS      FREQ_OBJ  FREQ_EFECTIVA
20ns         267492.790265  7.35    50.0MHz   79.0MHz   3.9961 mW
15ns         267380.182268  5.36    66.6MHz   103.7MHz  4.4172 mW
12ns         267372.675068  4.16    83.3MHz   127.5MHz  4.8507 mW
10ns         267372.675068  3.43    100.0MHz  152.2MHz  5.2843 mW
8ns          265628.502352  2.64    125.0MHz  186.5MHz  5.9354 mW
[synuser18@synopsys2 Reze]$
```

ppa_sweep.sh — un solo script itera los cinco periodos



Frecuencia objetivo vs. frecuencia real alcanzada

Corrección de PDK

Migración a una versión actualizada de Sky130 y re-ejecución del flujo completo

Corrección de PDK — migración

Mismo RTL, mismo diseño, PDK nuevo

- **Por qué cambio el PDK**

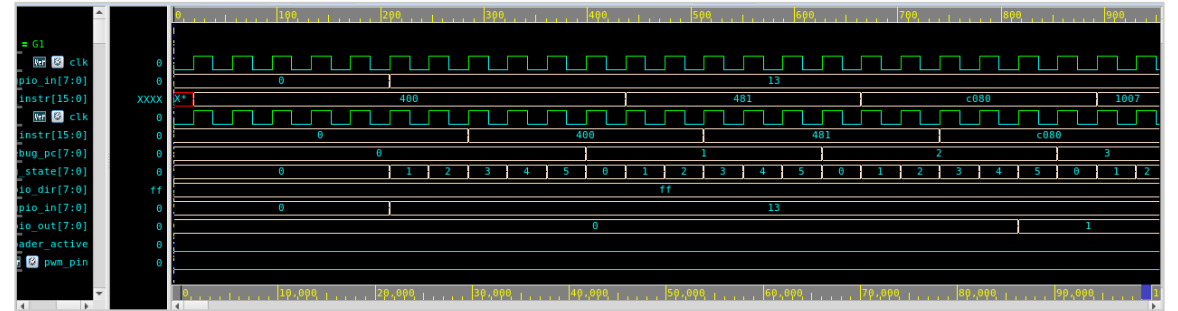
- El PDK original arrastraba errores muy sensibles como la falta de voltaje y gnd
- Se nos dio una versión actualizada del PDK Sky130, así que rehice el flujo desde cero con ella.

- **Cómo preparé el entorno**

- Partí de la plantilla del curso (04_full_cpu_project) y guardé su RTL de referencia en _generic_reference como un backup.
- Copié mis 10 módulos Verilog a rtl/ y mis testbenches a sim/.
- Adapté dc.tcl y signal_io.constraints, que estaban escritos para el CPU de la plantilla, para que apuntaran a microrv8_system.
- Guardé backups .generic_backup de cada script modificado.

- **Lo primero que verifiqué**

- Recompilar con VCS y volver a abrir la waveform en Verdi: si el RTL no simula igual, no tiene sentido seguir por que algo raro o malo había pasado



Waveform

```
pc = 7 instr = e007 state = 1 r1 = 1 r2 = 114 r5 = 0 r6 = 0 gpi
o_out = 1
pc = 7 instr = e007 state = 2 r1 = 1 r2 = 114 r5 = 0 r6 = 0 gpi
o_out = 1
pc = 7 instr = 090f state = 3 r1 = 1 r2 = 114 r5 = 0 r6 = 0 gpi
o_out = 1
$finish called from file "../sim/tb_system.sv", line 48.
$finish at simulation time 99010
V C S S i m u l a t i o n R e p o r t
Time: 99010
CPU Time: 0.500 seconds; Data structure size: 0.0Mb
Tue Aug 11 12:20:01 2026
lsynuser18@synopsys2 work sim1$
```

La simulación del sistema completo vuelve a correr en el entorno nuevo

Lo bueno de haber escrito scripts en vez de trabajar a mano: cambiar de PDK fue cambiar rutas y volver a lanzar, no rehacer el flujo.

Corrección de PDK — síntesis y layout

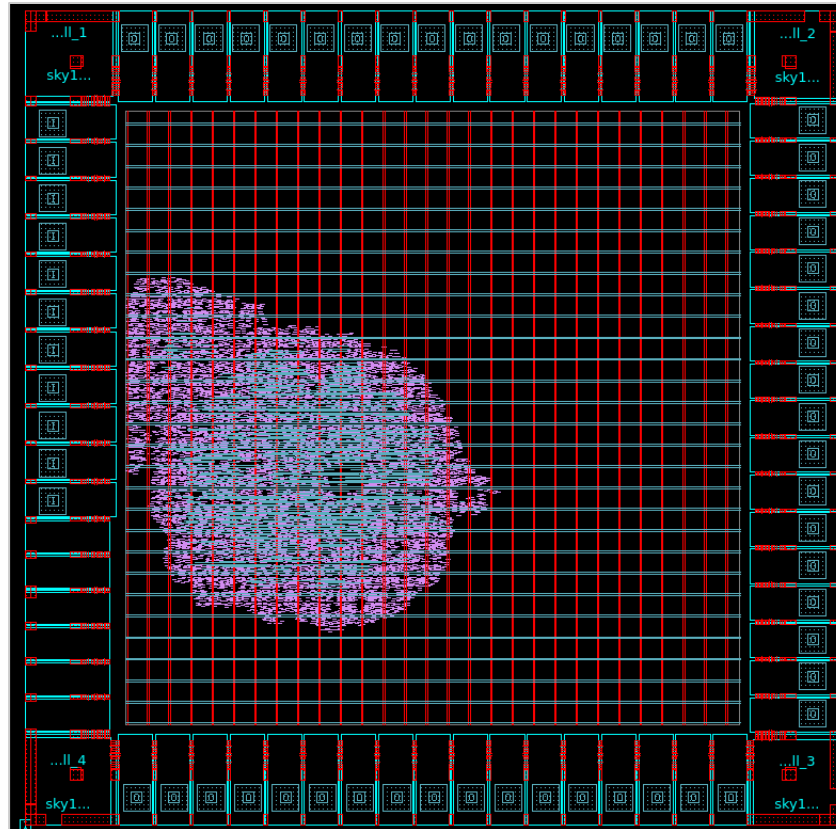
Design Compiler e IC Compiler II corriendo el mismo diseño sobre el PDK actualizado

```
=== QoR ===
[synuser18@synopsys2 04_full_cpu_project]
t | grep -A5 "Timing Path Group"
Timing Path Group 'INREG'
-----
Levels of Logic:                6.00
Critical Path Length:           1.33
Critical Path Slack:            12.97
Critical Path Clk Period:       20.00

Timing Path Group 'REGOUT'
-----
Levels of Logic:                1.00
Critical Path Length:           1.32
Critical Path Slack:            12.68
Critical Path Clk Period:       20.00

Timing Path Group 'clk'
-----
Levels of Logic:                18.00
Critical Path Length:           5.71
Critical Path Slack:            12.40
Critical Path Clk Period:       20.00
```

report_qor por timing path group tras compile_inc



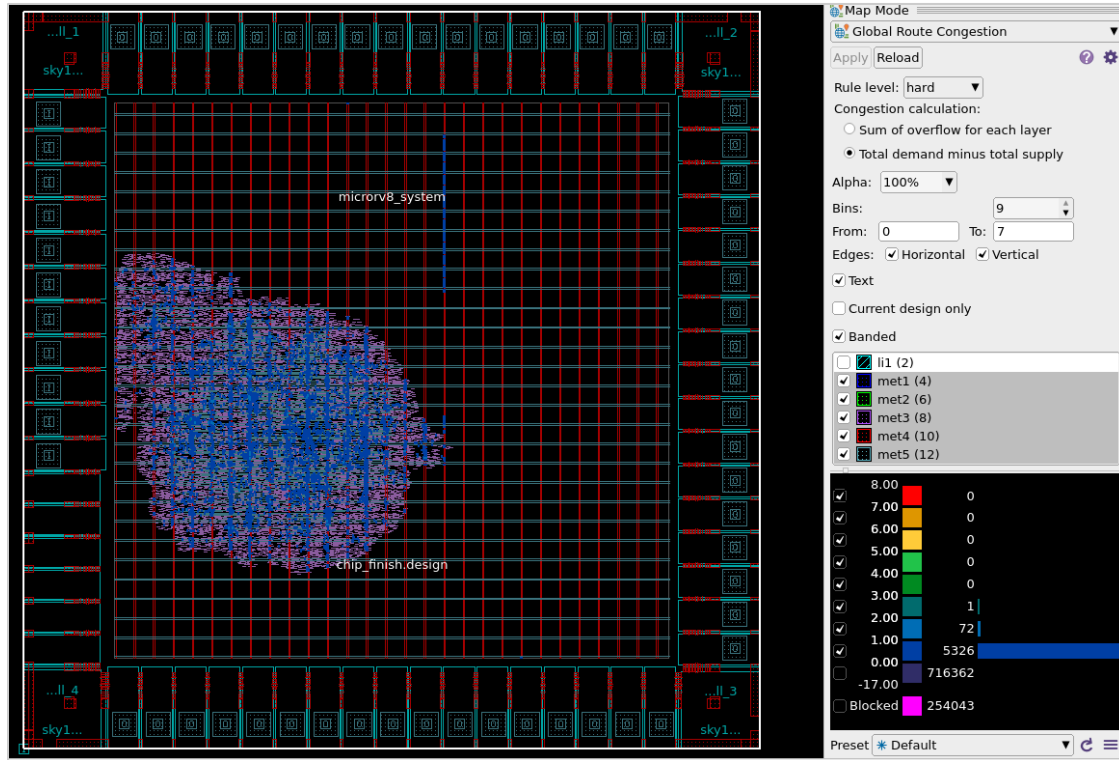
Layout final: bloque chip_finish en ICC2

Logical Hierarchy	Std	Hier Std	HM	Hier HM	Util	Pin	Net	Hier Net
chip_finish	50458	0	0	0.185	62		19636	
imem	15073	15585	0	0	70	15637	15637	
dmem	2088	2220	0	0	68	2259	2259	
cpu_clk_gate_pc_reg	1	1	0	0	6	4	4	
cpu_clk_gate_res_lat_reg	1	1	0	0	6	4	4	
cpu_clk_gate_alu_op_reg	1	1	0	0	6	4	4	
cpu_clk_gate_regs_reg_7	1	1	0	0	6	4	4	
cpu_clk_gate_regs_reg_6	1	1	0	0	6	4	4	
cpu_clk_gate_regs_reg_5	1	1	0	0	6	4	4	
cpu_clk_gate_regs_reg_4	1	1	0	0	6	4	4	
cpu_clk_gate_regs_reg_3	1	1	0	0	6	4	4	
cpu_clk_gate_regs_reg_2	1	1	0	0	6	4	4	
cpu_clk_gate_regs_reg_1	2	2	0	0	6	5	5	
cpu_clk_gate_gpio_out_reg	1	1	0	0	6	4	4	
cpu_clk_gate_rs1_addr_r_reg	2	2	0	0	6	5	5	
cpu_clk_gate_mem_addr_reg	1	1	0	0	6	4	4	
cpu_clk_gate_mem_wdata_reg	1	1	0	0	6	4	4	
gpio_clk_gate_gpio_out_reg	2	2	0	0	6	5	5	
gpio_clk_gate_gpio_dir_reg	2	2	0	0	6	5	5	
uart_clk_gate_tx_data_reg	1	1	0	0	6	4	4	
pwm_clk_gate_pwm_cnt_reg	1	1	0	0	6	4	4	
pwm_clk_gate_duty_reg	2	2	0	0	6	5	5	
pwm_clk_gate_prescaler_reg	2	2	0	0	6	5	5	
loader_clk_gate_state_reg	2	2	0	0	6	5	5	
loader_clk_gate_count_reg	1	1	0	0	6	4	4	
loader_clk_gate_total_reg	1	1	0	0	6	4	4	
loader_clk_gate_wr_addr_reg	1	1	0	0	6	4	4	
loader_clk_gate_instr_hi_reg	1	1	0	0	6	4	4	
loader_clk_gate_wr_data_reg	1	1	0	0	6	4	4	
uart_u_tx_clk_gate_bit_idx_reg	2	2	0	0	6	5	5	
uart_u_tx_clk_gate_shift_reg_reg	2	2	0	0	6	5	5	
uart_u_tx_clk_gate_baud_cnt_reg	1	1	0	0	6	4	4	
loader_u_rx_clk_gate_state_reg	2	2	0	0	6	5	5	
loader_u_rx_clk_gate_bit_cnt_reg	2	2	0	0	6	5	5	
loader_u_rx_clk_gate_data_out_reg	1	1	0	0	6	4	4	

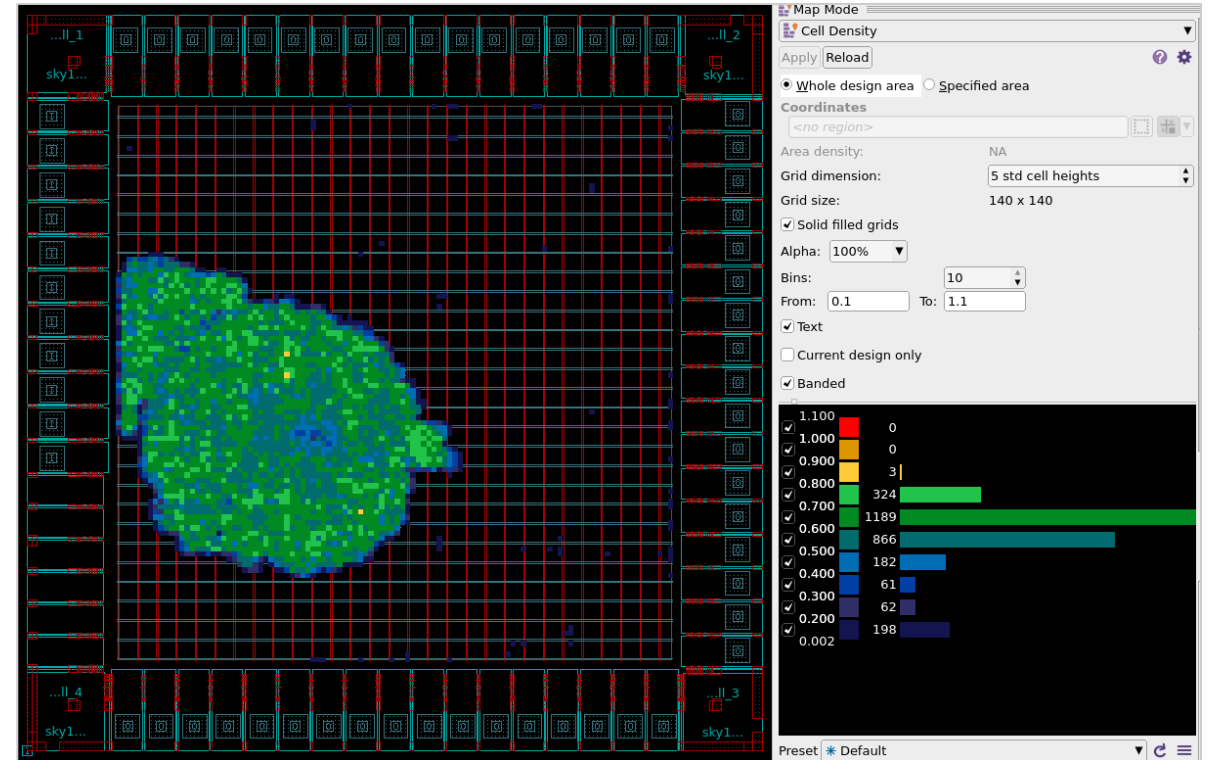
Jerarquía: 50,458 celdas std, util 0.185, imem y dmem dominando el area

Corrección de PDK — análisis del layout

Congestión, densidad de celdas y esquemático desde la GUI de IC Compiler II



Global route congestion sobre chip_finish



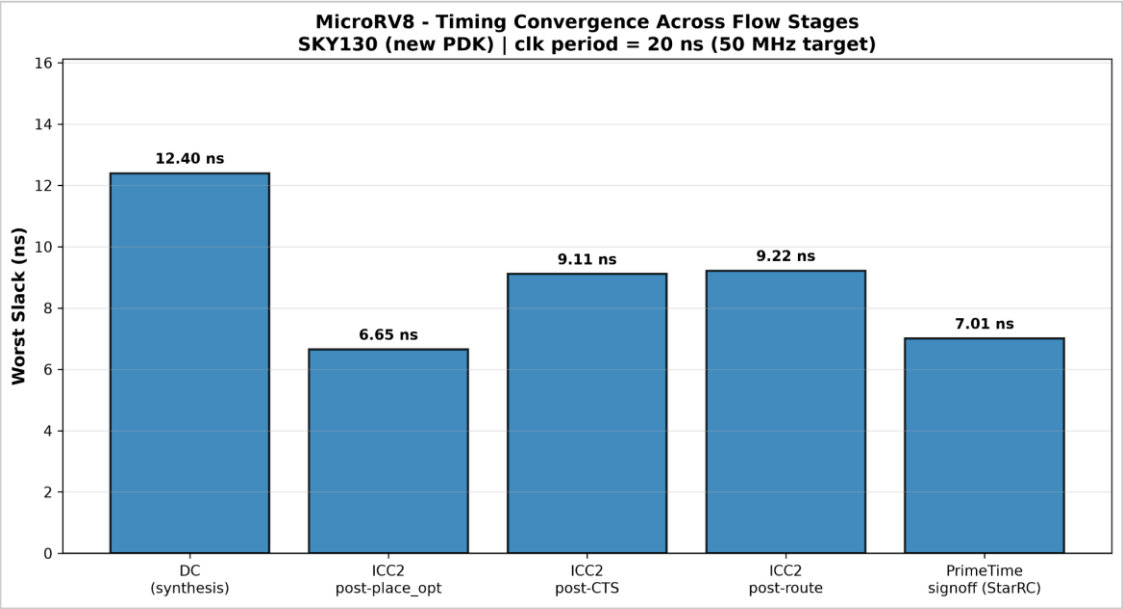
Cell density: el core se concentra en el centro del die

La utilización del core queda en 0.185, bastante baja, y eso se ve directamente en el mapa de densidad: hay mucho espacio libre alrededor del bloque. Es lo esperable en un diseño dominado por memorias (imem aporta 15,073 celdas de las 50,458 totales) y explica por qué la congestión nunca fue un problema real en este flujo. **El error browser de ICC2 no reporta violaciones.**

Corrección de PDK — PPA y convergencia de timing

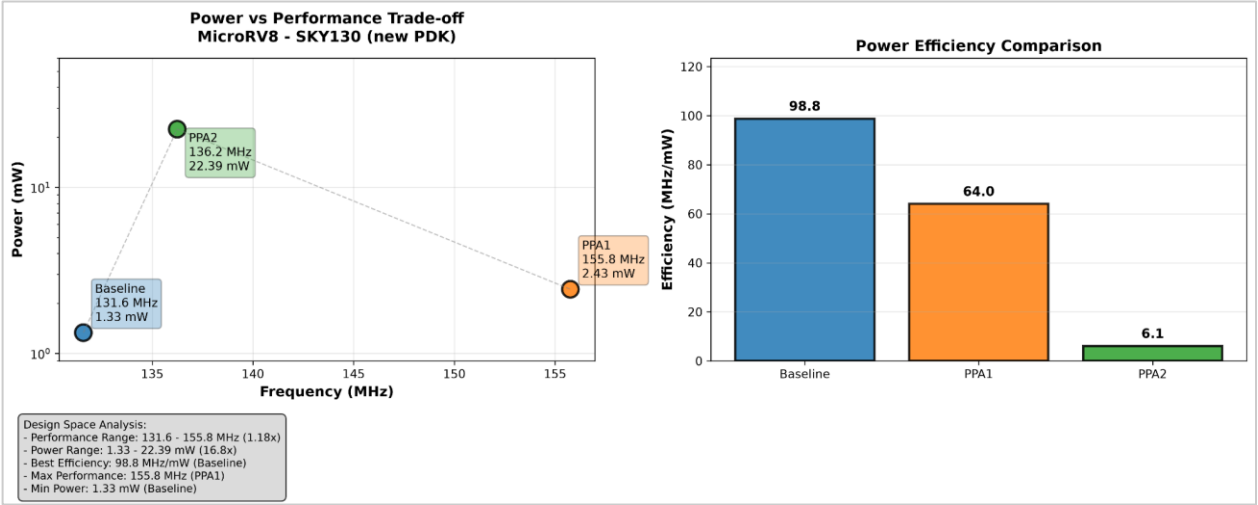
Dos exploraciones de PPA a partir de un SAIF generado con fsdb2saif

Convergencia de timing



Worst slack en cada etapa del flujo, periodo de 20 ns

Expectativa vs. resultado en las dos variantes



Power vs. performance y eficiencia de las tres corridas

Conclusiones

+7.01 ns

slack final en PrimeTime
sobre 20 ns (PDK nuevo)

1.33 mW

consumo del baseline
en el PDK actualizado

16.8×

de potencia que cuesta
quitar el clock gating

9,630 / 0

compare points passing
/ failing en Formality

Lo que se completó

El flujo entero, desde la simulación del RTL hasta un layout con timing cerrado: síntesis, DFT, verificación formal, place and route, extracción de parásitos y análisis estático de timing.

En el PDK original el diseño cierra en el corner típico con WNS de +4.05 ns y 3.99 mW gracias al clock gating. Tras migrar al PDK actualizado vuelve a cerrar sin violaciones: +7.01 ns de margen y 1.33 mW.

Único pendiente: PrimeTime ECO, por falta de licencia y comprensión

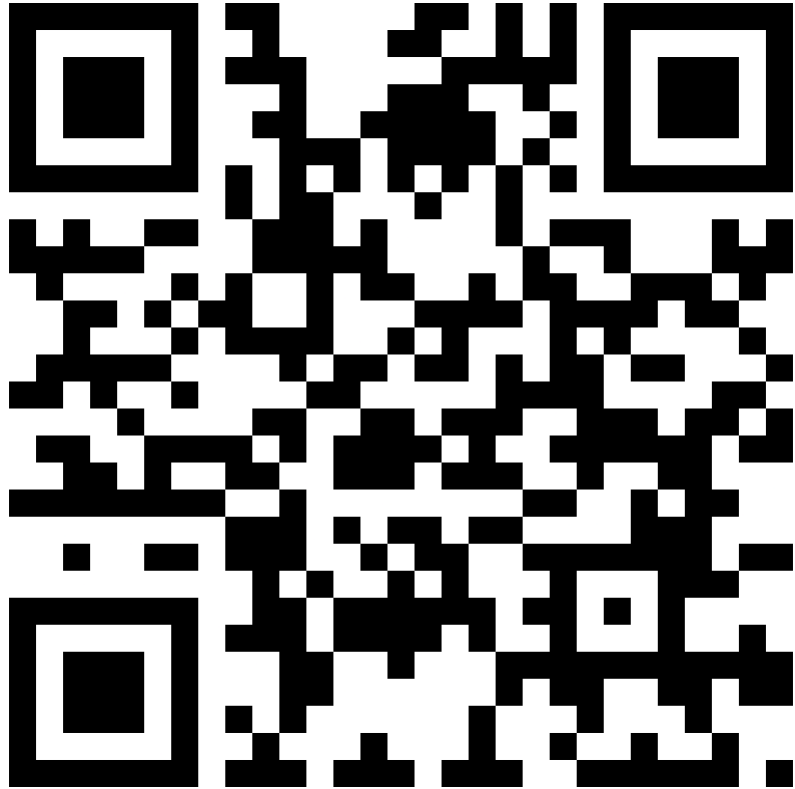
Lo que me llevo

- Verdi cambió cómo depuro. El bug de decodificación de tipo R no salía de un log ni de un print: salía de mirar la waveform y notar que el resultado no correspondía a los operandos con GTKWave lastimosamente no se ve muy bien
- El coverage bajo no siempre es un problema. El PWM lo tiene bajo por diseño, y entender por qué vale más que ir por el 100% siempre
- Lo más valioso fue llevar hasta layout un CPU que diseñé yo mismo, ya probado como softcore en una Tang Nano 9K, y comprobar que que si puede ser fabricado y validado

Referencias, links y reportes relevantes

Se añaden links a todo el trabajo que pude hacer y documentar

Repositorio de mi Softcore
(Falta actualizar)



Reportes hechos en fodisegua
Laboratorios y solucion – 300 paginas
Flujo completo – 143 paginas



SYNOPSYS®

Thank you